

Stored procedures

06_etapa2_procedures.sql — três rotinas no banco

PROCEDURE e FUNCTION não são a mesma coisa no PostgreSQL

O enunciado chama as três rotinas de stored procedures e dá nomes com prefixo `sp_`. No PostgreSQL, PROCEDURE e FUNCTION são objetos distintos, com formas de chamada e capacidades diferentes.

	PROCEDURE	FUNCTION
Como se chama	CALL nome(...)	SELECT nome(...) ou FROM nome(...)
Devolve valor	não, apenas parâmetros INOUT	sim
Devolve conjunto de linhas	não	sim, com RETURNS TABLE
Controle de transação	pode usar COMMIT e ROLLBACK	não pode

As duas rotinas que gravam dados são PROCEDURE. A que devolve um relatório é FUNCTION, porque precisa aparecer no FROM de um SELECT. O prefixo `sp_` foi mantido nos três nomes para seguir o enunciado.

sp_registrar_atendimento_completo

Grava um atendimento e todos os seus procedimentos em uma operação. A lista de procedimentos chega como array JSON, formato pedido pelo enunciado.

Chamada típica

```
CALL sp_registrar_atendimento_completo(  
  '2026-07-26 10:30',      -- data_hora  
  50,                    -- duracao_minutos  
  3, 13, 8, 2,           -- paciente, residente, preceptor, unidade  
  '[{"id_procedimento": 5, "tempo_real_minutos": 26,  
    "data_hora_inicio": "2026-07-26T10:36"},  
    {"id_procedimento": 2, "quantidade": 2, "tempo_real_minutos": 9}]::jsonb',  
  NULL                   -- INOUT: recebe o id gerado  
);
```

Só `id_procedimento` é obrigatório em cada item. `quantidade` assume 1, `faturado` assume falso, e `data_hora_inicio` assume o horário do próprio atendimento quando não vem informado. O último argumento é um parâmetro INOUT: o CALL devolve uma linha com o id do atendimento criado.

De onde vem a atomicidade

O enunciado pede que tudo seja revertido se qualquer procedimento falhar. O corpo da rotina não tem BEGIN nem COMMIT, e essa ausência é o ponto. Um CALL executa dentro da transação de quem chamou; qualquer exceção levantada no meio aborta o comando inteiro, desfazendo o INSERT do atendimento e os procedimentos já inseridos no laço. Um COMMIT no meio do corpo destruiria a garantia, porque confirmaria uma parte do trabalho antes de saber se o resto vai passar.

Quem chama precisa fazer o rollback. Na API isso está explícito: o handler captura DBAPIError, chama `db.rollback()` e traduz o erro em resposta HTTP.

Ordem das verificações

A rotina valida antes de gravar, em quatro etapas. Primeiro o formato do JSON, recusando o que não é array. Depois a lista não vazia: o DER exige no mínimo um procedimento por atendimento, cardinalidade que nenhuma constraint do esquema consegue expressar, porque envolve duas tabelas e o momento da inserção. Em seguida a existência de paciente, residente, preceptor e unidade. Por último, dentro do laço, a existência de cada procedimento.

As chaves estrangeiras recusariam as referências inválidas de qualquer forma. A verificação prévia existe para dizer qual delas está errada, em português, em vez de deixar subir a mensagem do driver.

Procedimento repetido na lista

Repetir o mesmo `id_procedimento` no array provoca violação da chave primária composta de `procedimento_realizado`, e o atendimento inteiro é desfeito. O comportamento é o correto: a repetição de um procedimento no mesmo atendimento se registra na coluna `quantidade`, não em duas linhas. Essa é também a forma mais rápida de demonstrar a reversão numa apresentação.

sp_calcular_tempo_medio_espera

Devolve, para cada unidade, a média do intervalo entre a chegada do paciente e o começo do primeiro procedimento daquele atendimento. É FUNCTION com RETURNS TABLE, porque o resultado é um conjunto de linhas.

Chamada

```
SELECT * FROM sp_calcular_tempo_medio_espera();
```

O cálculo tem duas etapas. Uma CTE encontra, por atendimento, o menor `data_hora_inicio` entre os procedimentos realizados. O SELECT externo tira a diferença em relação a `atendimento.data_hora`, converte para minutos com EXTRACT e agrupa por unidade.

Nomes das colunas de saída

As colunas devolvidas se chamam `unidade_id` e `nome_unidade`, e não `id_unidade` e `nome`. A diferença é técnica: em PL/pgSQL, um parâmetro de saída com o mesmo nome de uma coluna referenciada no corpo torna a referência ambígua, e a função falha em tempo de execução. Nomes que não colidem com nenhuma coluna do esquema evitam o problema sem precisar da diretiva `variable_conflict`.

Linhas que ficam de fora

- Atendimento sem unidade registrada, porque o agrupamento é por unidade.
- Atendimento sem nenhum procedimento com hora de início preenchida, porque não há como medir a espera.
- Procedimento cujo início consta como anterior à chegada do paciente. Isso indicaria dado errado, e uma única linha assim puxaria a média para baixo ou a deixaria negativa.

Resultado esperado com o seed

Unidade	Atendimentos	Espera média (min)
Enfermaria A	4	33,8

Unidade	Atendimentos	Espera média (min)
Ambulatorio	2	19,0
Pronto-Socorro	4	10,3
UTI Adulto	5	5,8

Os minutos do 09_etapa2_seed.sql foram escolhidos para que as unidades tenham esperas visivelmente diferentes, com a UTI atendendo quase na hora e a enfermaria demorando mais.

sp_reajustar_escala

Move os plantões de um residente de um dia e turno para outro, tudo ou nada.

Chamada

```
CALL sp_reajustar_escala(14, 'sexta', 'manha', 'quinta', 'manha', NULL);
```

O parâmetro INOUT devolve quantas escalas foram movidas. Zero quando não havia plantão na origem, caso em que a rotina emite um NOTICE e termina sem erro: pedir para mover algo que não existe é diferente de pedir algo impossível.

Como o conflito é detectado

Duas regras somadas garantem que um residente tenha no máximo um plantão em cada dia e turno. A UNIQUE(id_unidade, dia_semana, turno, id_residente) da Etapa 1 impede a repetição na mesma unidade. O trg_check_sobreposicao_escala da Etapa 2 impede a repetição em unidades diferentes. A rotina confere as duas antes de gravar, para poder explicar o motivo da recusa, e o trigger continua sendo a última barreira, inclusive contra quem escreve na tabela sem passar por aqui.

A verificação tem dois casos. Se mais de um plantão sair da origem para o mesmo destino, eles colidiriam entre si; se o destino já estiver ocupado pelo mesmo residente, colidiria com o que já existe. Nas duas situações a rotina levanta exceção antes de qualquer UPDATE, e nenhuma escala é movida pela metade.

Cada escala movida tem versao incrementada. A coluna é o contador do controle otimista da camada ORM: uma sessão que tenha carregado aquela escala antes do reajuste vai falhar ao gravar, em vez de sobrescrever a mudança.

Casos que o seed permite demonstrar

Chamada	Resultado
sp_reajustar_escala(14, 'sexta', 'manha', 'quinta', 'manha', NULL)	move 1 plantão; o residente 14 não tem nada na quinta
sp_reajustar_escala(12, 'quinta', 'manha', 'segunda', 'manha', NULL)	recusado: o residente 12 já tem plantão na segunda pela manhã
sp_reajustar_escala(11, 'quarta', 'noite', 'quinta', 'noite', NULL)	NOTICE e zero movidas: não há plantão na origem
sp_reajustar_escala(14, 'sexta', 'manha', 'sexta', 'manha', NULL)	recusado: origem e destino iguais

Como testar

A seção 7 e a seção 8 do 10_etapa2_verificacao.sql cobrem as três rotinas, com os resultados esperados em comentário e com ROLLBACK em volta das partes que gravam. Pela API, as duas rotinas de escrita estão em POST /etapa2/procedures/registrar-atendimento-completo e POST /etapa2/procedures/reajustar-escala, e a de leitura em GET /etapa2/procedures/tempo-medio-espera. Na interface, as duas primeiras ficam na aba Procedures e a terceira na aba Consultas ORM.